

2024-10-1

Thursday, September 25, 2025 3:06 PM

1. Considera la siguiente ecuación con $f: \mathbb{Z} \rightarrow \mathbb{Z}$:

$$f(x) = \begin{cases} x & \text{si } 10 \leq x \\ f(x+1) & \text{si } 0 < x < 9 \\ f(x-1) & \text{si } x \leq 0 \end{cases}$$

a) Proponé dos soluciones distintas en $\mathbb{Z} \rightarrow \mathbb{Z}$.

b) ¿Tiene una menor solución esa ecuación en $\mathbb{Z} \rightarrow \mathbb{Z}$?

c) ¿Si existe esa menor solución, podemos usar el teorema de menor punto fijo en $\mathbb{Z} \rightarrow \mathbb{Z}$ para encontrarla?

$$f(x) = \begin{cases} x & 10 \leq x \\ f(x+1) & 0 < x < 9 \rightarrow \text{tyerror} \\ f(x-1) & x \leq 0 \rightarrow \text{while} \end{cases}$$

$$g(x) = \begin{cases} x & \text{si } 10 \leq x \\ \text{tyerror} & \text{si } 0 < x \leq 9 \\ -10 & \text{si } x \leq 0 \end{cases}$$

supongamos que está mal y dice $0 < x \leq 9$

$$g(x) = \begin{cases} x & \text{si } 10 \leq x \\ 10 & \text{si } 0 < x \leq 9 \\ -10 & \text{si } x \leq 0 \end{cases}$$

↳ eso que podrias ponerle 4

b) no ya que $\mathbb{Z}$ no tiene minimo

c) Si existiera si podrias

2. Considera el lenguaje imperativo con input/output y fallas.

$$F: (\Sigma \rightarrow \Omega) \rightarrow (\Sigma \rightarrow \Omega)$$

$$F f \sigma = \begin{cases} \iota_{in} (z \mapsto \iota_{out} (z, f[\sigma | x: z])) & \text{si } \sigma x = 0 \\ \iota_{in} (z \mapsto \iota_{out} (z, f[\sigma | x: \sigma x - 1])) & \text{si } \sigma x > 0 \\ \iota_{abort} \sigma & \text{si } \sigma x < 0 \end{cases}$$

a) Proponé un programa de la forma **while** b **do** c cuyo funcional sea F .

b) Sea σ_k un estado tal que $\sigma_k x = k$. ¿Vale la igualdad $F^i \perp \sigma_2 = F^i \perp \sigma_3$ para todo i ?

c) ¿Hay algún estado σ tal que $F^2 \perp \sigma = \iota_{in} (z \mapsto \iota_{out} (z, \iota_{abort} [\sigma | x: z]))$?

a) (while $x \geq 0$ do ?z ; !z ; if $x=0$ then $x:=z$ else $x:=x-1$) ; fail
while true do if $x \geq 0$ then fail else (?z ; !z ; if ...)

b) $\sigma_k x = k$ $F^i \perp \sigma_3$

Vemos $F^i \perp \sigma_2$

$$\begin{aligned} F^1 \perp \sigma_2 &= \iota_{in} (z \mapsto \iota_{out} (z, \perp [\sigma | x: 1])) \\ &= \iota_{in} (z \mapsto \iota_{out} (z, \perp)) \end{aligned}$$

$$F^1 \perp \sigma_2 = \text{lin} (z_1 \mapsto \text{lat} (z_1, \perp [\sigma | x: 1]))$$

$$= \text{lin} (z_1 \mapsto \text{lat} (z_1, \perp))$$

$$F^2 \perp \sigma_2 = \text{lin} (z_1 \mapsto \text{lat} (z_1, \text{lin} (z_2 \mapsto \text{lat} (z_2, \perp [\sigma | x: 0])))$$

$$F^3 \perp \sigma_2 = \text{lin} (z_1 \mapsto \text{lat} (z_1, \text{lin} (z_2 \mapsto \text{lat} (z_2, \text{lin} (z_3 \mapsto \text{lat} (z_3, \perp [\sigma | x: z_3]))))$$

$$F^4 \perp \sigma_2 = \text{lin} (z_1 \mapsto \text{lat} (z_1, \text{lin} (z_2 \mapsto \text{lat} (z_2, \text{lin} (z_3 \mapsto \text{lat} (z_3, \text{F} \perp [\sigma | x: z_3]))))$$

$F^4 \perp \sigma_2$ te importa lo que introdujiste en z_3 supongamos $z_3 = -10$ podemos cortar el ciclo

$$F^1 \perp \sigma_3 = \text{lin} (z \mapsto \text{lat} (z, \perp [\sigma | x: 2]))$$

$$= \text{lin} (z \mapsto \text{lat} (z, \perp))$$

$$F^2 \perp \sigma_3 = \text{lin} (z_1 \mapsto \text{lat} (z_1, \text{lin} (z_2 \mapsto \text{lat} (z_2, \perp [\sigma | x: 1])))$$

$$F^3 \perp \sigma_3 = \text{lin} (z_1 \mapsto \text{lat} (z_1, \text{lin} (z_2 \mapsto \text{lat} (z_2, \text{lin} (z_3 \mapsto \text{lat} (z_3, \perp [\sigma | x: 0]))))$$

aca nos va a tener uno mas pero se ve igual

c) *no depende solo del estado, con σ_1 puede llegar a tener la F.*

3. Considera la expresión $e = \lambda y. y ((\lambda x. x) y)$. Respondé si es verdadero, falso, o *impreciso* cada ítem justificando tu respuesta.

- En el calculo lambda puro, $e(ue)$ tiene forma normal. Aquí u es una variable cualquiera.
- En el calculo con evaluación normal, para cualquier expresión e' existe z (forma canónica) tal que $ee' \Rightarrow_N z$.
- En el calculo con evaluación eager, $e \Rightarrow_E (\lambda y. y y)$ y por lo tanto ee no tiene forma canónica.

$$e = \lambda y. y ((\lambda x. x) y) = \lambda y. y y$$

$$e(ue) = (ue) ((\lambda x. x) (ue))$$

$$= (ue) (ue) \quad \text{forma normal.}$$

b) falso si $e' = e$

$$ee \Rightarrow_N ee \Rightarrow_N^* ee \text{ nunca llega a un } \perp$$

c) creo que falso porque $e \Rightarrow_E e$ por regla

e es tomado como valor hasta que se aplique

lo segundo es verdadero pero la implicación empieza con algo falso.

4. Considera los lenguajes aplicativos eager y normal.

- Considera la siguiente ecuación recursiva $t = \iota_{\text{tuple}} \langle 0, t \rangle$. Proponé una expresión en el lenguaje aplicativo normal cuya semántica sea una solución para t .
- Considera la expresión $\text{letrec } f \equiv \lambda u. \langle 0, \lambda v. f \ 0 \rangle \text{ in } f$. Calcula la semántica eager de dicha expresión; para ello, primero expresá el funcional asociado de la manera más sencilla posible.

5. Proponé una expresión e que tenga como única variable libre a n tal que $\llbracket e \rrbracket [\eta \mid n : \iota_{\text{int}} k]$ sea un estado con $\max(k, 0)$ referencias todas accesibles a partir del valor devuelto. Por ejemplo, con $k = 1$ la semántica podría ser $\iota_{\text{norm}} \langle [r_0 : \iota_{\text{tuple}} \langle \rangle], \iota_{\text{ref}} r_0 \rangle$.

4) a. $t = \iota_{\text{tuple}} \langle 0, t \rangle$

lenguaje aplicativo $\rightarrow$ letrec

$$\longrightarrow \text{rec } \lambda t. \langle 0, t \rangle$$

$$\xrightarrow{\text{eager}} \text{letrec } t \equiv \lambda t. \langle 0, t \rangle \text{ in } t$$

b. $\llbracket \text{letrec } f \equiv \lambda u. \langle 0, \lambda v. f \ 0 \rangle \text{ in } f \rrbracket \eta$

$$= \llbracket f \rrbracket [\eta \mid f : \iota_{\text{fun}} g]$$

$$g = Y F$$

$$F h \perp = \llbracket \langle 0, \lambda v. f \ 0 \rangle \rrbracket [\eta \mid f : \iota_{\text{fun}} h, u : \perp]$$

$$\llbracket \text{letrec } v \equiv \lambda u. e_0 \text{ in } e \rrbracket \eta = \llbracket e \rrbracket [\eta \mid v : \iota_{\text{fun}} g]$$

donde g es el menor punto fijo de

$$F f \perp = \llbracket e \rrbracket [\eta \mid v : \iota_{\text{fun}} f, u : \perp]$$

o sea

$$g = Y_{V_{\text{fun}}} F \quad Y_{V_{\text{fun}}} F = \sqcup_i F^i \perp$$

$$g = Y_{V_{\text{fun}}} (\lambda f \in V_{\text{fun}}. \lambda z \in V. \llbracket e \rrbracket [\eta \mid v : \iota_{\text{fun}} f, u : z])$$

$$\begin{aligned}
&= (\lambda z_1. (\lambda z_2. \text{tuple}(\langle z_1, z_2 \rangle)) \cdot \llbracket \lambda v. f\ 0 \rrbracket \eta') \cdot \llbracket 0 \rrbracket \eta') \\
&= \lambda z_2. \text{tuple}(\langle \text{lint } 0, z_2 \rangle) \cdot \llbracket \lambda v. f\ 0 \rrbracket \eta' \\
&= \lambda z_2. \text{tuple}(\langle \text{lint } 0, z_2 \rangle) \cdot (\text{fun}(\lambda v_2. \overbrace{\llbracket f\ 0 \rrbracket [\eta \mid v: v_2]}^{\eta''})) \\
&= \lambda z_2. \text{tuple}(\langle \text{lint } 0, z_2 \rangle) \cdot (\text{fun}(\lambda v_2. (\lambda g. g \text{ lint } 0)_{\text{fun}} \llbracket f \rrbracket \eta'')) \\
&= \lambda z_2. \text{tuple}(\langle \text{lint } 0, z_2 \rangle) \cdot (\text{fun}(\lambda v_2. (h(\text{lint } 0)))) \\
&= \text{tuple} \langle \text{lint } 0, \text{fun}(\lambda v_2. h \text{ lint } 0) \rangle
\end{aligned}$$

$$\begin{aligned}
F^1 \perp &= \text{tuple} \langle \text{lint } 0, \text{fun}(\lambda v_2. \perp \text{ lint } 0) \rangle \\
&= \text{tuple} \langle \text{lint } 0, \text{fun}(\lambda v_2. \perp) \rangle
\end{aligned}$$

$$F^2 \perp = \text{tuple} \langle \text{lint } 0, \text{fun}(\lambda v_2. F \perp \text{ lint } 0) \rangle$$

$$\begin{aligned}
F^2 \perp &= \text{tuple} \langle \text{lint } 0, \text{fun}(\lambda v_2. \text{tuple} \langle \text{lint } 0, \text{fun}(\lambda v_2. \perp \text{ lint } 0) \rangle) \rangle \\
&\equiv \text{tuple} \langle \text{lint } 0, \text{fun}(\lambda v_2. \text{tuple} \langle \text{lint } 0, \text{fun}(\lambda v_2. \perp) \rangle) \rangle
\end{aligned}$$

5. Proponé una expresión e que tenga como única variable libre a n tal que $\llbracket e \rrbracket [\eta \mid n : \text{lint } k]$ sea un estado con $\max(k, 0)$ referencias todas accesibles a partir del valor devuelto. Por ejemplo, con $k = 1$ la semántica podría ser $\text{tnorm} \langle [r_0 : \text{tuple} \langle \rangle], \text{tref } r_0 \rangle$.

$$\llbracket e \rrbracket [\eta \mid n : \text{lint } k]$$

$\max(k, 0)$ referencias

$$k=1 \rightarrow \text{tnorm} \langle [r_0 : \text{tuple} \langle \rangle], \text{tref } r_0 \rangle$$

$$\text{letrec } f \equiv \lambda i. \quad \begin{array}{ll} \text{if } i \leq 0 \text{ then } 0 & \text{in } f\ n \\ \text{else if } i = 0 \text{ then ref } 0 & \\ \text{else let } z \equiv f(i-1) \text{ in ref } z & \end{array}$$

```

else if i = 0 then ref 0
else let z ≡ f(i-1) in ref z

```



```

ref z
z → ref z'
z' → ref... z''
z'' → ref 0

```

yo queria poner
 ref 0; ref z pero chatGPT
 me bardeo por que sino pierdo
 la primera ref y me
 piden que sea accesible

```

letrec f ≡ λi. if i ≤ 0 then <> in fn
else let x = f(i-1) in
let r = ref 0 in
<r, x>

```

```

letrec f ≡ λi. if i ≤ 0 then <> in fn
else <ref 0, f(i-1)>

```

solución
 más pensable.

2024-02-07

Thursday, September 25, 2025 9:30 PM

1. Considerá la siguiente ecuación recursiva.

$$g(x) = \begin{cases} 1 & \text{si } x = 0 \\ g(|x| - 2) & \text{si } x \neq 0 \end{cases}$$

Sea $F: (\mathbb{Z} \rightarrow \mathbb{Z}_{\perp}) \rightarrow \mathbb{Z} \rightarrow \mathbb{Z}_{\perp}$ definido por:

$$F f x = \begin{cases} x + 1 & \text{si } x \in \{0, 1\} \\ f(|x| - 2) & \text{si } x \notin \{0, 1\} \end{cases}$$

- a) Escribí de la manera más clara posible $\sqcup_k (F^k \perp)$.
b) ¿Es ese supremo una solución para g ? Justificá tu respuesta.

$$F^1 \perp = \begin{cases} x+1 & x \in \{0, 1\} \\ \perp & \end{cases}$$

$$F^2 \perp = \begin{cases} x+1 & x \in \{0, 1\} \\ \begin{cases} |x|-1 & |x|-2 \in \{0, 1\} = \{2, 3, -2, -3\} \\ \perp & \text{c.c.} \end{cases} \end{cases}$$

$$F^3 \perp = \begin{cases} x+1 & x \in \{0, 1\} \\ \begin{cases} |x|-1 & x \in \{2, 3, -2, -3\} \\ \begin{cases} ||x|-2|-1 & x \in \{-4, -5, 4, 5\} \\ \perp & \text{c.c.} \end{cases} \end{cases} \end{cases} \quad ||x|-2| - 2 \in \{0, 1\}$$

$$= \begin{cases} 1 & x=0 \\ 2 & x=1 \\ 1 & 2 \\ 2 & 3 \\ 1 & 4 \\ 2 & 5 \end{cases}$$

$$\sqcup_k F^k \perp = \begin{cases} 1 & |x| \text{ es par} \\ 2 & |x| \text{ es impar } \wedge \neq 1 \\ \perp & x = -1 \end{cases}$$

Si es supremo ya que respeta que los pares devuelven 1.

2. Considerá el lenguaje imperativo con fallas, input y output. Sea c el comando $x := 1$; **while** $x > 0$ **do** $!x$.

Sea $\omega_0, \omega_1, \omega_2, \dots$ la cadena en Ω tal que $\llbracket c \rrbracket = \sqcup_i (\omega_i)$.

- a) Escribí explícitamente los valores de ω_i para $i < 5$.
b) Sea ω'_j una cadena en Ω tal que $\omega'_i = \omega_i$ si $i < 5$ pero que $\omega'_6 = \omega'_5$. Proponé un comando c' cuya semántica sea $\sqcup_j (\omega'_j)$.

$$\llbracket c \rrbracket \sigma = \llbracket x := 1; \text{while } x > 0 \text{ do } !x \rrbracket \sigma \\ = \llbracket \text{while } x > 0 \text{ do } !x \rrbracket \underbrace{\llbracket \sigma / x:1 \rrbracket}_{\sigma'}$$

$$w_0, \dots \text{ en } \Omega \text{ tal q } \llbracket c \rrbracket = \bigcup_i (w_i)$$

$$\llbracket x := 5; \text{while } x > 0 \text{ do } !1; x := x-1 \rrbracket$$

$$Fw \sigma' = \begin{cases} w_* \llbracket !x \rrbracket \sigma' & \sigma' x > 0 \\ \sigma' & \sigma' x = 0 \end{cases}$$

$$w^0 = F^0 \perp \sigma' = \perp$$

$$w^1 = F^1 \perp \sigma' = \bigcup_{\sigma} \llbracket !x \rrbracket \sigma' \text{ si true}$$

$$= \perp_* (\text{last } \llbracket !x \rrbracket \sigma', \text{term } \sigma') \leftarrow$$

$$= (\text{last } \perp, \perp_* \text{term } \sigma')$$

$$= (\text{last } \perp, \perp)$$

$$w^2 = F^2 \perp \sigma' = F(F \perp) \sigma'$$

$$= F_* (\text{last } \llbracket !x \rrbracket \sigma', \text{term } \sigma')$$

$$= (\text{last } \perp, F_* \text{term } \sigma')$$

$$= (\underbrace{\text{last } \perp}_{1'}, (\text{last } \perp, \perp))$$

$$w^4 = (1', (1', (1', (1', \perp))))$$

3. Considera el cálculo lambda puro. Proponé una expresión e que tenga distintas formas canónicas en los órdenes de evaluación. Justificá tu respuesta haciendo las evaluaciones.

$$e = (\lambda x. (\lambda y. (\lambda z. z) y)) 1$$

$$\boxed{1 = \lambda f_x. f_x}$$

no por que en λ eager se evalúa
directa.

$$\rightarrow (\lambda x. (\lambda z. x)) ((\lambda x. x) (\lambda y. y))$$

$$\Rightarrow_{\beta} \lambda z. ((\lambda x. x) (\lambda y. y))$$

$$\Rightarrow_{\epsilon} \lambda z. (\lambda y. y)$$

4. Considera el lenguaje eager con recursión y la expresión

$$e = \lambda y. \text{letrec } f \equiv \lambda x. \text{if } x < y \text{ then } x \text{ else } f(x-y) \text{ in } f$$

a) Evalúa e 5 9.

b) ¿Cuál es la semántica denotacional de e (-3) 3?

$$(\lambda y. \text{letrec } f \equiv \lambda x. [\text{if } x < y \text{ then } x \text{ else } f(x-y)] \text{ in } f) 5 9$$

$$\lambda y. \dots \Rightarrow \lambda y. \dots$$

$$5 \Rightarrow 5$$

$$(\text{letrec } f \equiv \lambda x. [\text{if } x < 5 \text{ then } x \text{ else } f(x-5) \text{ in } f]) 9$$

$$\rightarrow \text{letrec } f \equiv \dots$$

$$f / f \mapsto \lambda x. \text{letrec } f \equiv \lambda x. e \text{ in } e$$

$$\equiv \lambda x. \text{letrec } f \equiv \lambda x. e \text{ in } e$$

$$\Rightarrow //$$

$$\rightarrow 9 \Rightarrow 9$$

$$\rightarrow \text{letrec } f \equiv \lambda x. e \text{ in } e / x \mapsto 9$$

$$\equiv \text{letrec } f \equiv \lambda x. e \text{ in } \text{if } 9 < 5 \text{ then } 9 \text{ else } f(9-5)$$

$$e' / f \mapsto \lambda x. \text{letrec } f \equiv x. e \text{ in } e$$

$$\equiv \text{if } 9 < 5 \text{ then } 9 \text{ else } (\lambda x. \text{letrec } f \equiv x. e \text{ in } e)(9-5)$$

$$9 < 5 \Rightarrow \text{False}$$

$$\Rightarrow (\lambda x. \text{letrec } f \equiv x. e \text{ in } e)(9-5)$$

$$\begin{aligned}
 & \parallel \quad | \Rightarrow (\lambda x. \text{letrec } f \equiv x.e \text{ in } e)(4-5) \\
 & \parallel \quad \left| \begin{array}{l} \lambda x... \Rightarrow \lambda x... \\ 4-5 \Rightarrow 4 \\ \lambda x... / x \mapsto 4 \\ \equiv \text{letrec } f \equiv \lambda x.e \text{ in } \overbrace{\text{if } 4 < 5 \text{ then } 4 \text{ else } f(4-5)}^{e''} \end{array} \right. \\
 & \parallel \quad \left| \begin{array}{l} e'' / f \mapsto \lambda x.e \\ \equiv \text{if } 4 < 5 \text{ then } 4 \text{ else } \lambda x.e(4-5) \end{array} \right. \\
 & \parallel \quad \left| \begin{array}{l} 4 < 5 \Rightarrow \text{True} \\ \Rightarrow 4 \end{array} \right. \\
 & \Rightarrow 4.
 \end{aligned}$$

$$\begin{aligned}
 \text{b) } & \llbracket e(-3)3 \rrbracket \eta \\
 & = (\lambda f \in V_{\text{fun}}. f \llbracket 3 \rrbracket \eta)_{\text{fun}^*} \llbracket e(-3) \rrbracket \eta \\
 & = \underbrace{(\lambda f \in V_{\text{fun}}. f 3)_{\text{fun}^*}}_{f'} \underbrace{(\lambda f \in V_{\text{fun}}. f \llbracket -3 \rrbracket \eta)_{\text{fun}^*}}_{f''} \llbracket e \rrbracket \eta \\
 & = f' f'' \llbracket e \rrbracket \eta \\
 & = f' f'' \llbracket \lambda y. \text{letrec } f \equiv \lambda x. [\text{if } x < y \text{ then } x \text{ else } f(x-y)] \text{ in } f \rrbracket \eta \\
 & = f' f'' \text{ (fun } (\lambda y' \in V. \underbrace{\llbracket \text{letrec } \dots \rrbracket \llbracket \eta \mid y: y' \rrbracket}_{\eta'}) \\
 & = *
 \end{aligned}$$

$$\llbracket \text{letrec } f \equiv e \text{ in } f \rrbracket \eta' = \llbracket f \rrbracket [\eta' / f : g]$$

$$\text{donde } g = YF$$

$$\begin{aligned}
 Ff'z & = \llbracket \text{if } x < y \text{ then } x \text{ else } f(x-y) \rrbracket \overbrace{[\eta' / f : f', x : z]}^{\eta''} \\
 & = (\lambda b \in V_b \{ \llbracket x \rrbracket \eta'' \quad b \})_{\text{fun}^*} \llbracket x < y \rrbracket \eta''
 \end{aligned}$$

$$= \left(\lambda b \in V_b \begin{cases} \llbracket x \rrbracket \eta'' & b \\ \llbracket f(x-y) \rrbracket \eta'' & \neg b \end{cases} \right)_{\text{bool}^*} \llbracket x < y \rrbracket \eta''$$

$$= \begin{cases} \llbracket z \rrbracket \eta'' & (z < y') \\ \llbracket f(x-y) \rrbracket \eta'' & \neg(z < y') \end{cases} = \textcircled{*}'$$

$$\begin{aligned} \llbracket f(x-y) \rrbracket \eta'' &= \left(\lambda f'' \in V_f. f'' \llbracket x-y \rrbracket \eta'' \right)_{\text{func}} \llbracket f \rrbracket \eta'' \\ &= f'(\text{lnt}(z-y')) \end{aligned}$$

$$\textcircled{*}' = \begin{cases} \text{lnt } z & (z < y') \\ f'(\text{lnt}(z-y')) & \text{cc} \end{cases}$$

$$\text{vemos } F^0 \perp = \perp$$

$$F^1 \perp = \begin{cases} \text{lnt } z & (z < y') \\ \perp & \text{cc} \end{cases}$$

$$F^2 \perp = \begin{cases} \text{lnt } z & (z < y') \\ \begin{cases} \text{lnt}(z-y') & (z-y' < y') \\ \perp & (\text{c.c.}) \end{cases} & \end{cases}$$

$$F^3 \perp = \begin{cases} \text{lnt } z & z < y' \\ \begin{cases} \text{lnt}(z-y') & z-y' < y' \\ \begin{cases} \text{lnt}((z-y')-y') & z-2y' < y' \\ \perp & \end{cases} \end{cases} & \end{cases}$$

$$g = \begin{cases} \text{lnt } z - ky' & \exists \text{ min } k \text{ positivo tq } z-ky' < y' \\ \perp & \text{si no existe } k. \end{cases}$$

$$\textcircled{*} = f' f'' g$$

$$= g(-3) z = \perp //$$

5. Considera el lenguaje eager con referencias. Proponé una expresión e_1 y otra expresión e_2 tales que:

$\llbracket \text{val}(\text{ref } e_1) \rrbracket \sigma = \iota_{\text{norm}}(\sigma', \iota_{\text{int}} 2)$ y

$\llbracket \text{val}(\text{ref } e_2) \rrbracket \sigma = \text{tyerr}$

$$e_1 = 2$$

$$e_2 = \text{True} + 3$$

- Definí las nociones de predominio y de dominio.
- Definí la noción de función continua.
- Definí el funcional asociado a `while b do c` y demostrá que es continuo.

Sea $(C, \perp)$ un poset

decimos que es un predominio si toda cadena interesante tiene supremo

monotone $\rightarrow$ orden
continua $\rightarrow$ supremo
estricto $\rightarrow$ minimo

dominio es un predominio que tiene minimo.

b) una función es continua si preserva supremos

c) `while b do c` =

$$Ff\sigma = \begin{cases} \sigma & \neg [b]\sigma \\ f_{\perp}([c]\sigma) & [b]\sigma \end{cases}$$

2. Considerá el lenguaje imperativo con input/output y fallas. Sea F el funcional asociado a

`while  $x \neq y$  do ! $x$ ; ? $y$`

- Definí de la manera más sencilla posible F .
- ¿Es $F^* \perp$ una cadena interesante?

$[\text{while } x \neq y \text{ do } !x; ?y] \sigma$

$$Ff\sigma = \begin{cases} \sigma \\ f_{*} [!x; ?y] \sigma \end{cases} = \begin{cases} \sigma \\ f_{*} g \end{cases}$$

$$\begin{aligned} [!x; ?y] \sigma &= [!y]_{*} ([!x] \sigma) \\ &= [!y]_{*} (l_{\text{out}}([!x] \sigma, l_{\text{term}} \sigma)) \\ &= l_{\text{out}}([!x] \sigma, [!y]_{*} l_{\text{term}} \sigma) \\ &= l_{\text{out}}([!x] \sigma, l_{\text{in}}(\lambda y'. l_{\text{term}}([?y; y]))) \end{aligned}$$

b) si es una cadena interesante por p.e. al sumarle un $F_{\perp}$ cambiando.

3. Considerá la expresión $e = \lambda y. \lambda x. y y x$. Respondé si es verdadero o falso cada ítem justificando tu respuesta.

- En el calculo lambda puro, para toda expresión e' se tiene que ee' tiene forma normal.
- En el calculo lambda normal, para toda expresión e' se tiene que existe z (forma canónica) tal que $ee' \Rightarrow z$.
- En el calculo lambda eager, para toda expresión e' se tiene que existe z (forma canónica) tal que $ee' \Rightarrow z$.

$$e = \lambda y. \lambda x. y y x$$

a) falso $e \Delta y$ no tiene

- T
- F

$$1) C) [\text{while } e \text{ do } c] \sigma = Y F \sigma$$

$$Fw\sigma = \begin{cases} w_{\perp} & [e]\sigma \\ \sigma & \neg [e]\sigma \end{cases}$$

$F \in (E \rightarrow E_{\perp}) \rightarrow (E \rightarrow E_{\perp})$ continuo por todo c, e

sea $f_1, f_2, \dots$ donde $E \rightarrow E_{\perp}$

$$\text{vemos que } \bigcup_{i=0} F f_i = F \left(\bigcup_{i=0} f_i \right)$$

$$\text{por def vemos que } \bigcup_{i=0} F f_i \sigma = F \left(\bigcup_{i=0} f_i \right) \sigma$$

caso $[e]\sigma$ y $[c]\sigma \neq \perp$

$$\bigcup F f_i \sigma = \bigcup (f_i)_{\perp} ([c]\sigma) = \bigcup f_i ([c]\sigma)$$

$$F \left(\bigcup f_i \right) \sigma = \left(\bigcup f_i \right)_{\perp} ([c]\sigma) = \bigcup f_i ([c]\sigma)$$

caso $[e]\sigma$ y $[c]\sigma = \perp$

$$\bigcup F f_i \sigma = \bigcup f_i_{\perp} [c]\sigma = \bigcup \perp$$

$$F \left(\bigcup f_i \right) \sigma = \left(\bigcup f_i \right)_{\perp} [c]\sigma = \perp$$

igualdad su valor de variable.

4. Considera el lenguaje eager con recursión y la expresión

$$e = \lambda y. \text{letrec } f \equiv \lambda x. \text{if } x > y \text{ then } y \text{ else } f(x+y) \text{ in } f$$

a) Evalúa $e \ 2 \ -3$.

b) ¿Cuál es la semántica denotacional de $e \ 3 \ 3$?

$e \ 2 \ -3$

$(\lambda y. \text{letrec } f = \lambda x. \text{if } x > y \text{ then } y \text{ else } f(x+y) \text{ in } f) \ 2 \ -3$

$$\lambda y. \dots \rightarrow \lambda y. \dots$$

$$2 \Rightarrow 2$$

$$(\text{letrec } f \equiv \lambda x. \text{if } x > 2 \text{ then } 2 \text{ else } f(x+2) \text{ in } f) (-3)$$

$$(\text{letrec } f \equiv \dots)$$

$$\Rightarrow f / f \mapsto \lambda x. \text{letrec } f \equiv \dots \text{ in } e$$

$$\lambda x. \text{letrec } f \equiv e \text{ in } e$$

$$-3 \Rightarrow -3$$

$$\text{letrec } f \equiv \lambda x. e \text{ in if } -3 > 2 \text{ then } 2 \text{ else } f(-3+2)$$

$$\text{if } -3 > 2 \text{ then } 2 \text{ else } f \mapsto \lambda x. \text{let} \dots$$

$$-3 > 2 \Rightarrow \text{false}$$

$$(\lambda x. \text{letrec } f \equiv \lambda x. e \text{ in if } x > 2 \text{ then } 2 \text{ else } f(x+2)) (-1)$$

⋮

2 //

$$\llbracket \text{letrec } f \equiv \lambda x. e' \text{ in } e \rrbracket \sigma = \llbracket e \rrbracket [\sigma | f : g]$$

$$g = \lambda f. F_h z = \llbracket e \rrbracket [\sigma | f : h | x : z]$$

5. Proponé una expresión e que tenga como única variable libre a n tal que $\llbracket e \rrbracket [\eta | n : \iota_{\text{int}} k] \rrbracket$ sea un estado con $|k|$ referencias todas accesibles a partir del valor devuelto. Por ejemplo, con $k = 1$ la semántica podría ser $\iota_{\text{norm}} ([r_0 : \iota_{\text{tuple}} \langle \rangle], \iota_{\text{ref}} r_0)$.

$$n \quad \llbracket e \rrbracket [\eta | n : \iota_{\text{int}} k] \rrbracket \quad |k|$$

$$\text{letrec } f \equiv \lambda x. \text{if } x \leq 0 \text{ then } \langle \rangle \text{ in } f_k$$

$$\text{else } \langle \text{ref}_0, f(x-1) \rangle$$

letrec is wim.

$$\begin{aligned} & \llbracket \text{letrec } v \equiv \lambda u. e \text{ in } e' \rrbracket \eta \theta \\ &= \llbracket e' \rrbracket [\eta / v : r_0] [\theta' / r_0 : \text{fun } g] \theta \end{aligned}$$

X

$w: \text{fun } g$

$r_0 = \text{new}(\theta')$

$$g = YF$$

$$Fh z = \llbracket e \rrbracket [\eta / u : r_0] [\theta' / r_0 : z]$$

$$Ff \langle \theta', z \rangle = \llbracket e \rrbracket [\eta / w : \text{fun } f] \theta'$$

$$\llbracket \text{letrec } w = \lambda u. e \text{ in } e' \rrbracket \theta =$$

$$= \llbracket e' \rrbracket [\theta / w : \text{fun } g]$$

$$g = YF$$

$$Ff z = \llbracket e \rrbracket [\theta / w : f / u : z]$$

$$\llbracket \text{letrec } w = \lambda u. e \text{ in } e' \rrbracket \eta \theta = \llbracket e' \rrbracket [\eta / w : \text{fun } g] \theta$$

$$\llbracket \text{letrec } w = \lambda u. e \text{ in } e' \rrbracket \eta \theta = \llbracket e' \rrbracket [\eta / w: \text{fun } g] \theta$$

$$F \vdash \langle \theta', z \rangle = \llbracket e \rrbracket [\eta / w: \text{fun } f / u: z] \theta'$$

L imp Simp $\llbracket \text{newvar } v = e \text{ in } e' \rrbracket \theta$

$$= \left(\text{Rest}_{v\theta} \right) \llbracket e' \rrbracket [\theta / v = \llbracket e \rrbracket \theta]$$

LIS falls $\downarrow$
+

LIS LIS + iof LAE LAN Isw

$$\begin{array}{lcl} CL & \longrightarrow & D_{\infty} \\ & \searrow & \\ & \longrightarrow & E \\ & \searrow & \\ & \longrightarrow & N \end{array}$$

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \eta \theta =$$

$$\llbracket c \rrbracket [\eta / v: \text{let } r_0] [\theta' / r_0: e']$$

→ *actualize nuevo θ'
 e' este valor.*

donde $\llbracket e \rrbracket \eta \theta = \text{letrec } \langle \theta', e' \rangle$ $r_0 = \text{new}(\text{dom}(\theta'))$